PROJECT DESIGN PHASE-II

Solution Requirements (Functional and Non-functional)

Date	25 October 2026
Team ID	Individual project
Project Name	HotelPro (Hotel Management System)
Maximum Marks	4 Marks

Functional requirements

FR No.	Functional requirement (epic)	Sub-requirements (stories / sub-tasks)
FR-1	User authentication and roles	Sign up with name, email and password. Log in and receive a token-based (JWT) session. Passwords stored hashed. Three roles: guest, staff, admin. Public sign-up always creates a guest.
FR-2	Room management (admin)	Add, edit and delete rooms with number, type, price, capacity, description and amenities. Set a room to maintenance. Deleting a room that has active bookings is blocked.
FR-3	Room search and availability	Search by check-in, check-out, number of guests and room type. Only rooms free on every requested night are listed. The check-out day is free for the next guest.
FR-4	Booking management	Create a booking with server-side price calculation (nights x nightly rate). Reject past dates, over-capacity requests and double bookings. View own bookings. Cancel an upcoming booking.
FR-5	Payments	Pay online by UPI, card or net banking (simulated gateway, transaction reference issued). Pay at hotel, recorded by staff. A paid booking that is cancelled is marked refunded.
FR-6	Front-desk operations	List all bookings with a status filter. Check a guest in (from the arrival date) and out. Mark cash payments as paid.
FR-7	Dashboard and reports	Show guests in house, arrivals today, departures due, occupancy percentage, rooms under maintenance, registered guests and revenue collected. Show the guest list with booking counts.

Non-functional requirements

NFR No.	Non-functional requirement	Description
NFR-1	Usability	Clean, responsive interface built with Bootstrap that works on phones, tablets and desktops. Clear error and empty-state messages. Keyboard focus is visible on all controls.
NFR-2	Security	Passwords hashed with bcrypt. JWT authentication with role-based authorization on every protected route. HTTP security headers (helmet), CORS allow-list, rate limiting on login and sign-up, input type checks against injection, request size limit. No card data is collected or stored.
NFR-3	Reliability	Server validates all dates, capacity and status changes. Booking states follow fixed rules (confirmed, checked-in, checked-out, cancelled). Errors return a consistent JSON message and never expose stack traces in production.
NFR-4	Performance	Target: pages load within 2 seconds on a normal connection and API responses return within 500 ms. Database indexes on bookings (room and dates) and users (email) support these targets.
NFR-5	Availability	The API is stateless, so it can be restarted or run in several copies. Target uptime depends on the hosting plan; free tiers may sleep when idle.
NFR-6	Scalability	Stateless JWT sessions, a REST API and MongoDB allow horizontal scaling to support high booking volumes. Dates and prices are handled in one place so features can be added without redesign.
NFR-7	Maintainability	Modular structure (models, routes, middleware, utilities) with a README that documents setup, API routes and deployment.

PROJECT DESIGN PHASE-II

Data Flow Diagram and User Stories

Date	25 October 2026
Team ID	Individual project
Project Name	HotelPro (Hotel Management System)
Maximum Marks	4 Marks

Data flow diagrams

A data flow diagram (DFD) shows how data moves through HotelPro: who sends data in, which processes handle it, and where it is stored. Level 0 shows the whole system as one process. Level 1 breaks it into its main processes and data stores.

Level 0: context diagram

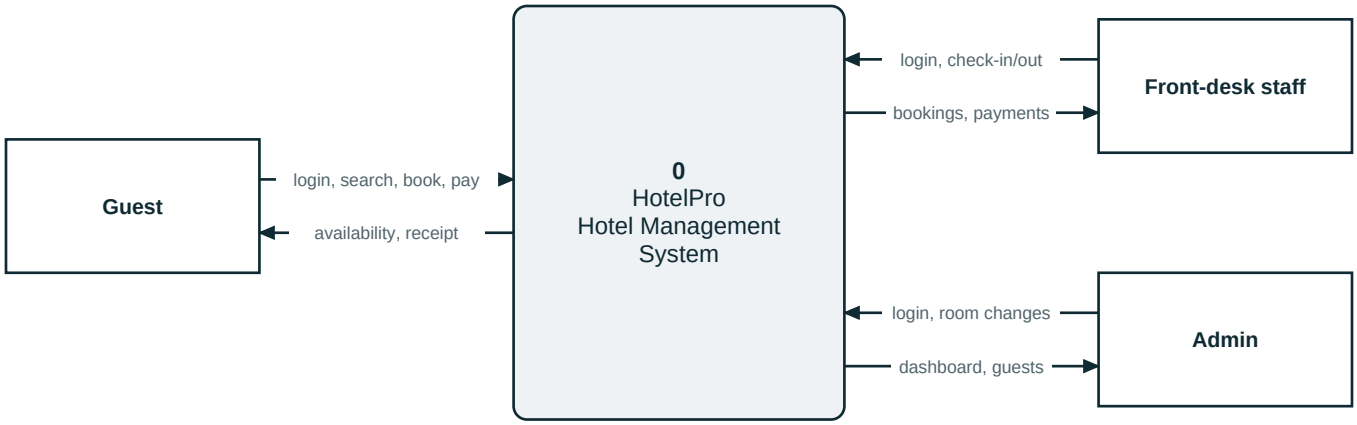

Figure 1. Level 0 DFD of HotelPro.

Level 1: main processes

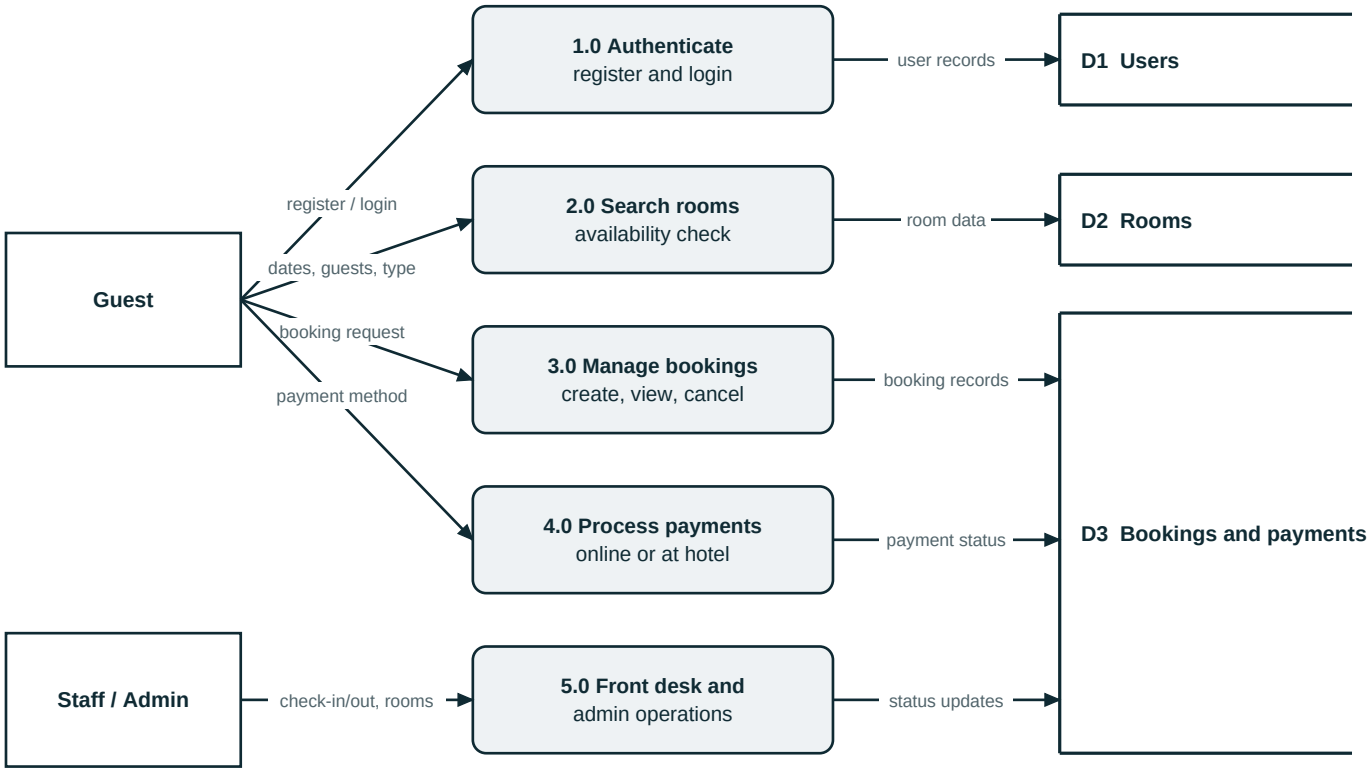

Figure 2. Level 1 DFD. Staff and admin also sign in through process 1.0. Responses flow back to the same actors along each path.

Process	Input	Output	Data stores used
1.0 Authenticate	Name, email, password	JWT token and user role	D1 Users
2.0 Search rooms	Check-in, check-out, guests, type	List of rooms free on those nights	D2 Rooms (read), D3 Bookings (read)
3.0 Manage bookings	Room, dates, guests, special requests	Confirmed booking with total price. Cancellation result	D2 Rooms (read), D3 Bookings (write)
4.0 Process payments	Payment method, or staff cash entry	Transaction reference, paid or refunded status	D3 Bookings and payments (write)
5.0 Front desk and admin operations	Check-in, check-out, room changes, report request	Updated statuses, dashboard figures, guest list	D1, D2, D3 (read and write as allowed by role)

User stories

User type	Functional requirement (epic)	User story / task	Acceptance criteria	Priority	Release
Guest	FR-1 Authentication	As a guest, I can create an account and log in.	Account is created with a hashed password and I receive a session. Wrong credentials show a clear error.	High	Sprint-1
Guest	FR-3 Search	As a guest, I can search rooms by dates, guests and room type.	Only rooms free on every selected night and with enough capacity are listed.	High	Sprint-2
Guest	FR-4 Booking	As a guest, I can book a room for my dates.	Total price is nights x nightly rate. The booking appears in My bookings as confirmed.	High	Sprint-2
Guest	FR-4 Booking	As a guest, I cannot book a room that is already reserved.	A booking that overlaps an existing stay is rejected with a clear message.	High	Sprint-2
Guest	FR-4 Booking	As a guest, I can cancel an upcoming booking.	Status becomes cancelled and the room is free again. A paid booking is marked refunded.	Medium	Sprint-2
Guest	FR-5 Payments	As a guest, I can pay for my booking online.	Payment status becomes paid and a transaction reference is shown.	High	Sprint-3
Staff	FR-6 Front desk	As front-desk staff, I can check a guest in and out.	Check-in works only for confirmed bookings from the arrival date. Check-out works only for checked-in guests.	High	Sprint-3
Staff	FR-6 Front desk	As front-desk staff, I can see all bookings and filter by status.	List shows guest, room, dates, total, status and payment status.	High	Sprint-3
Staff	FR-5 Payments	As front-desk staff, I can record a pay-at-hotel payment.	Payment status becomes paid with a cash reference.	Medium	Sprint-3
Admin	FR-2 Rooms	As an admin, I can add, edit, delete and close rooms.	Changes are saved. A room with active bookings cannot be deleted.	High	Sprint-1
Admin	FR-7 Dashboard	As an admin, I can see occupancy, arrivals and revenue.	Figures match the bookings and payments stored.	High	Sprint-4
Admin	FR-7 Dashboard	As an admin, I can see the list of registered guests.	List shows name, email, phone, number of bookings and last stay.	Medium	Sprint-4

PROJECT DESIGN PHASE-II

Technology Stack (Architecture and Stack)

Date	25 October 2026
Team ID	Individual project
Project Name	HotelPro (Hotel Management System)
Maximum Marks	4 Marks

Technical architecture

HotelPro uses a scalable three-tier architecture: a presentation layer (React frontend), a business-logic layer (Node.js and Express REST API) and a data layer (MongoDB). Layers communicate only through the REST API, which keeps them independent and easy to scale.

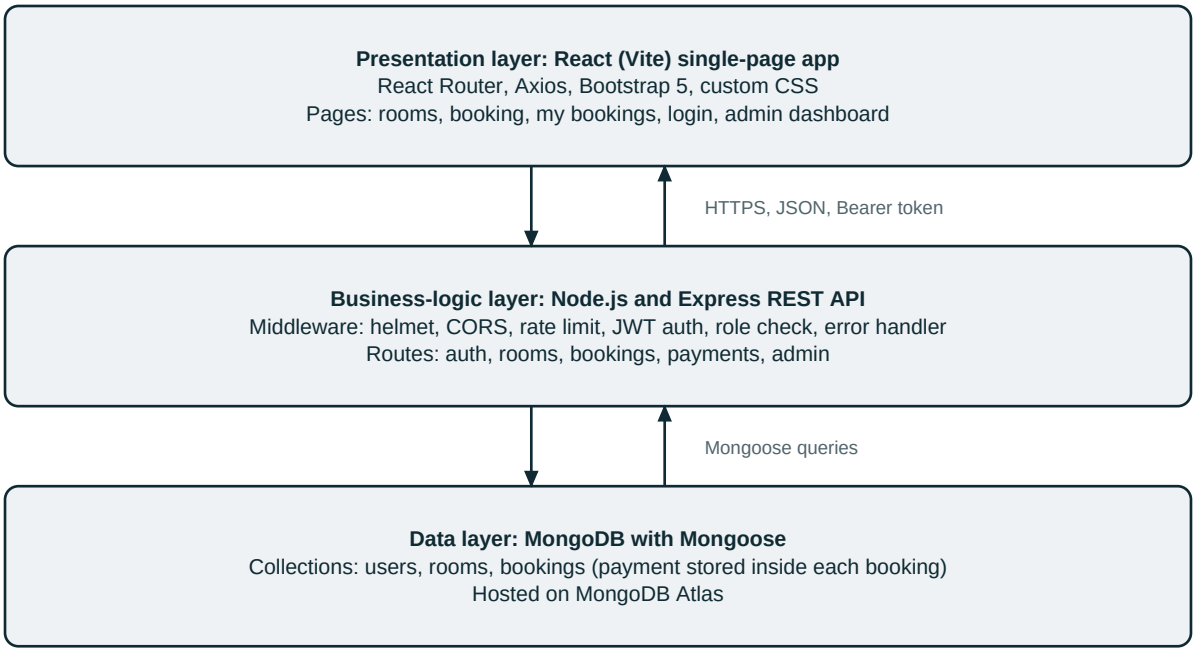

Figure 3. Three-tier architecture of HotelPro.

Table 1: Components and technologies

S.No	Component	Description	Technology
1	User interface	Responsive web interface for guests, staff and admin	HTML, CSS, JavaScript, React JSX, Bootstrap
2	Application logic 1	Authentication, rooms, availability search, bookings, payments	Node.js, Express.js
3	Application logic 2	Front-desk operations, dashboard statistics, guest list	Node.js, Express.js
4	Authentication	Token-based sessions and password hashing	JSON Web Token (JWT), bcrypt
5	Database	Stores users, rooms, bookings and payment details	MongoDB, Mongoose
6	Asynchronous processing	Non-blocking API calls and database queries	Async/await (asynchronous JavaScript), Axios
7	Payment	Simulated gateway; ready to replace with a live provider	Custom module (Razorpay or Stripe in future)
8	Hosting	Frontend, backend and database hosting	Vercel or Netlify, Render, MongoDB Atlas

Table 2: Application characteristics

S.No	Characteristic	Description	Technology
1	Open-source frameworks	Frontend and backend frameworks	React.js, Node.js, Express.js, Bootstrap, Mongoose
2	Security implementation	Hashed passwords, role-based access, secure headers, rate limiting	bcrypt, JWT, helmet, express-rate-limit, CORS
3	Scalable architecture	Three-tier design with a stateless RESTful API (MVC style)	REST, MongoDB indexes, stateless JWT
4	Availability	Cloud-hosted services that can be restarted or scaled	Render, Vercel, MongoDB Atlas
5	Performance	Indexed queries and lightweight single-page frontend	MongoDB indexes, Vite build

References

- React documentation: react.dev
- Node.js documentation: nodejs.org
- Express: expressjs.com
- Mongoose: mongoosejs.com
- MongoDB Atlas: mongodb.com/atlas